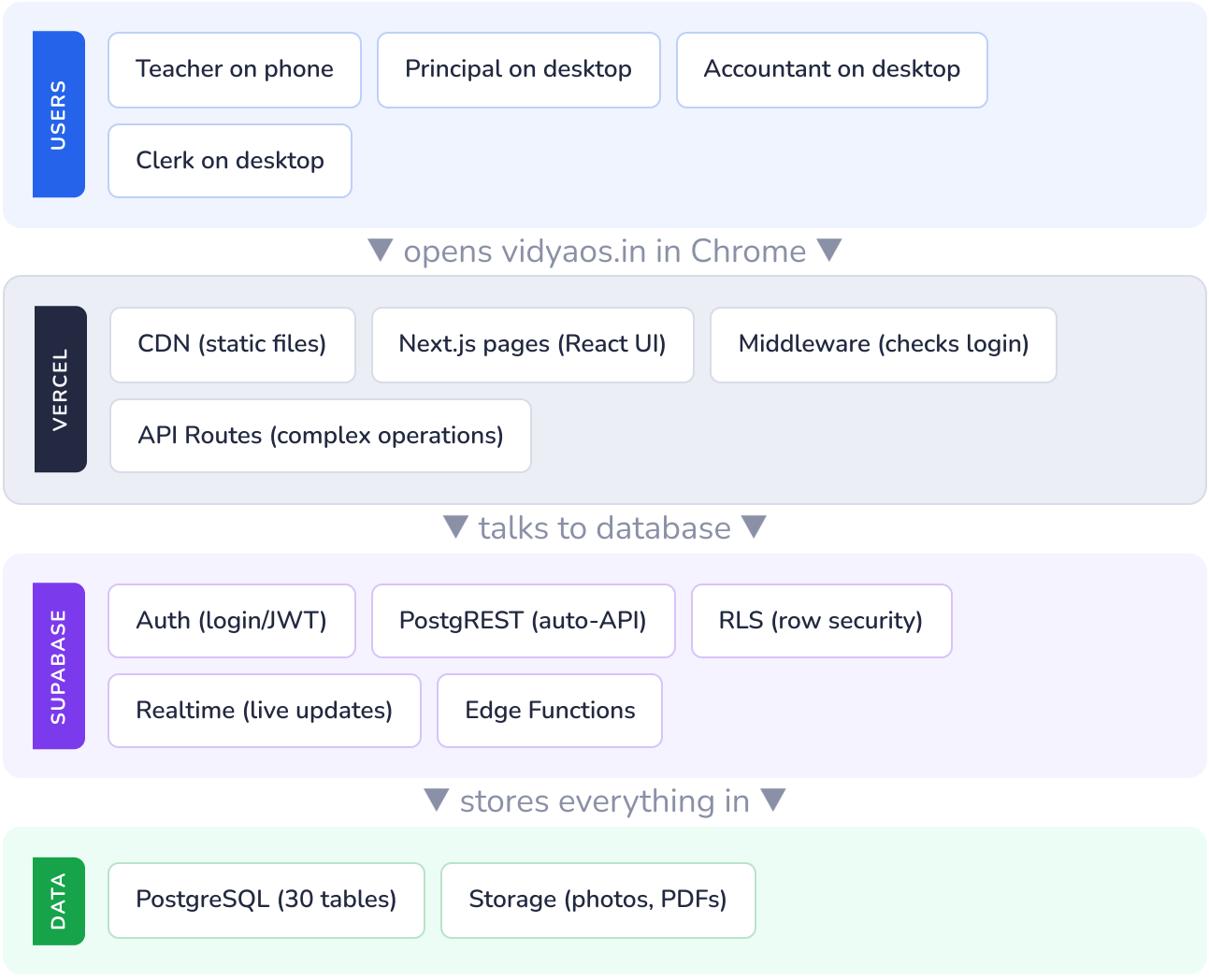
VidyaOS — How It All Works

Architecture + API guide for Guru Nanak Public School's management system. Each section shows what happens, where in the system it happens, and the exact code that makes it work.

NEXT.JS + SUPABASE | FREE TIER — ₹42/MONTH | 30 TABLES · 6 ROLES

The big picture — 4 layers

Every action in VidyaOS passes through these layers. Think of it like a sandwich — each layer has a specific job.



Two ways data travels

This is the most important concept. Every API call in VidyaOS uses one of these two paths.

Path A — Direct SDK (80% of calls)

The browser talks directly to Supabase. The user's login token (JWT) goes with every request. Row Level Security (RLS) in the database decides what data they can see.

- Reading student lists
- Marking attendance
- Entering exam marks
- Viewing fee ledger
- Posting notices
- Uploading photos

Path B — API Routes (20% of calls)

The browser calls a Vercel function first, which then talks to Supabase with a master key. Used when the operation is too complex or sensitive for the browser.

- Creating user accounts
- Bulk importing from Excel
- Generating PDF receipts
- Generating report cards
- Promoting students to next class
- Data exports

💡 **Why two paths?** Path A is faster and costs nothing (no server involved). But some operations need a "master key" that can't be exposed to the browser — like creating accounts or bypassing security to do bulk inserts. That's when Path B kicks in.

Login & Account Creation

How users get into the system and how accounts are managed

No real emails needed. Teachers log in with a username like "raman_sir". Internally we store it as `raman_sir@vidyaos.local` — a fake email that Supabase Auth requires. This saves us from needing email verification or SMS OTPs.

1 Admin creates a teacher account

POST /API/AUTH/CREATE-USER

The principal or clerk fills a form with username, password, name, and role. The server creates the account using a master key.

Path B — API Route

principal clerk

```
// Request
{
  "username": "raman_sir",
  "password": "Temp@1234",
  "full_name": "Raman Singh",
  "role": "class_teacher"
}

// What the server does:
// 1. Creates auth user: raman_sir@vidyaos.
// 2. Sets role in JWT: { role: "class_teac
// 3. Inserts row in 'profiles' table
// 4. Inserts row in 'staff' table

// Response — 201
{
  "user_id": "a1b2c3d4-...",
  "username": "raman_sir",
  "message": "Share password verbally."
}
```

2 Teacher logs in

Teacher enters username + password on the login page. Supabase Auth verifies and returns a JWT token stored as an HTTP-only cookie (secure, can't be stolen by JS).

Path A — Direct SDK

all roles

SUPABASE.AUTH.SIGNINWITHPASSWORD()

```
const { data, error } = await supabase
  .auth.signInWithPassword({
    email: `${username}@vidyaos.local`,
    password: password
  })

// Returns JWT with claims:
// {
//   sub: "a1b2c3d4-...",
//   app_metadata: {
//     role: "class_teacher",
//     school_id: "sch_001"
//   },
//   exp: 1726574400  // 1 hour
// }

// Token auto-refreshes every hour.
// Stored in HTTP-only cookie.
```

3 Middleware checks every page load

Before any page renders, Edge Middleware reads the cookie, validates the JWT, and checks if this

MIDDLEWARE.TS (RUNS ON EVERY REQUEST)

```
// Pseudocode:
```

role can access this page. Unauthorized? Redirected to login.

Runs on Vercel Edge — automatic

```
const user = await getUser(cookie)

if (!user) redirect('/login')

const role = user.app_metadata.role

// Route protection:
// /settings/users → principal only
// /fees/*          → accountant, principal
// /attendance/*    → class_teacher, princi
// /dashboard       → principal only

if (!canAccess(role, pathname)) {
  redirect('/unauthorized')
}
```

Marking Attendance

The most-used feature — a teacher marks attendance on their phone every morning

1 Teacher opens attendance page

Selects their class (auto-filled since they're assigned to one) and today's date. The system fetches the student list.

Path A — Direct SDK

class_teacher

FETCH STUDENTS FOR THIS CLASS

```
const { data: students } = await supabase
  .from('students')
  .select(`
    id, first_name, last_name,
    enrollment:student_class_enrollments!in
      id, roll_number
    )
  `)
  .eq('enrollment.class_id', myClassId)
  .eq('enrollment.academic_year_id', curren
  .order('enrollment.roll_number')

// RLS ensures this teacher can ONLY
// see students in their assigned class.
// A teacher assigned to Class 5-A
// cannot query Class 6-B students.
```

2 Teacher marks each student

BATCH UPSERT ATTENDANCE

A toggle switch next to each name: Present (green) / Absent (red) / Late (yellow). Then hits Submit.

Path A — Direct SDK

```
const { error } = await supabase
  .from('attendance_records')
  .upsert(
    students.map(s => ({
      student_id: s.id,
      enrollment_id: s.enrollment.id,
      date: '2026-09-27',
      status: s.markedStatus,
      // 'present'|'absent'|'late'|'half_da
      subject_id: null,
      // null = daily, set for subject-wise
      marked_by: currentUserId,
      school_id: schoolId
    })),
    {
      onConflict: 'student_id,date,subject_
      // If already marked today, UPDATE in
    }
  )
// ✅ 201 Created → show success toast
```

3 How RLS protects this

The database itself checks: "Is this teacher assigned to the class these students belong to?" If not, the INSERT is rejected with a 403 error. No server code needed for this check.

RLS POLICY ON ATTENDANCE_RECORDS

```
CREATE POLICY "teachers_mark_own_class"
ON attendance_records FOR INSERT
WITH CHECK (
  -- Check 1: same school
  school_id = current_user_school_id()
  -- Check 2: student is in teacher's class
  AND enrollment_id IN (
    SELECT sce.id
    FROM student_class_enrollments sce
    JOIN class_teacher_assignments cta
      ON cta.class_id = sce.class_id
      AND cta.section_id = sce.section_id
    WHERE cta.teacher_id = auth.uid()
  )
);
```

4 View attendance reports

Monthly summary is pre-computed as a database VIEW — percentage per student. Used for defaulter flagging and report cards.

Path A — Direct SDK

READ MONTHLY SUMMARY VIEW

```
const { data } = await supabase
  .from('v_monthly_attendance_summary')
  .select('*')
  .eq('class_id', classId)
```

class_teacher principal

```
  .eq('month', '2026-09')

// Returns per student:
// {
//   student_name: "Harpreet Kaur",
//   present: 22, absent: 3, late: 1,
//   total_days: 26,
//   percentage: 84.6
// }
// Students below 75% get flagged.
```

Fee Collection & Receipts

Accountant collects fees, generates receipts, and tracks defaulters

1 Look up a student's fee ledger

Accountant searches by admission number. The ledger shows all charges for the year — tuition, transport, lab fee — and what's paid vs pending.

Path A — Direct SDK

accountant principal

QUERY FEE LEDGER WITH JOINS

```
const { data } = await supabase
  .from('student_fee_ledger')
  .select(`
    id, period, amount, concession_amount,
    net_amount,    -- GENERATED: amount - c
    paid_amount,
    balance,       -- GENERATED: net - paid
    status,        -- pending | paid | over
    due_date,
    fee_head:fee_heads(name)
  `)
  .eq('student_id', studentId)
  .eq('academic_year_id', currentYear)
  .order('due_date')

// net_amount and balance are PostgreSQL
// generated columns — always accurate,
// no app-layer math bugs possible.
```

2 Record a payment

Accountant fills: amount, mode (cash/UPI), and selects which fee items this payment covers. Uses a database function (RPC) to ensure the payment + ledger update is atomic.

SUPABASE.RPC('RECORD_FEE_PAYMENT')

```
const { data } = await supabase
  .rpc('record_fee_payment', {
    p_student_id: 'stu_001',
    p_amount: 3500,
```

Database Function (RPC)

accountant

```
    p_mode: 'cash', // cash|upi|bank|cheque
    p_reference: null, // UPI ref / cheque
    p_items: [
      { ledger_id: 'led_01', amount: 1500 }
      // Sept Tuition
      { ledger_id: 'led_02', amount: 500 },
      // Sept Transport
      { ledger_id: 'led_03', amount: 1500 }
      // Oct Tuition
    ]
  })

// Returns:
// { payment_id: "pay_042",
//   receipt_number: "GNPS/2026-27/0042" }
// Receipt # is auto-generated by PostgreSQ
// sequence — unique and gap-free.
```

3 Generate receipt PDF

After payment, accountant clicks "Print Receipt". A Vercel serverless function queries the data, generates a PDF with school letterhead, and uploads it to Storage.

Path B — API Route

accountant

POST /API/RECEIPTS/GENERATE

```
// Request
{ "payment_id": "pay_042" }

// Server-side steps:
// 1. Verify user is accountant (from JWT)
// 2. Query payment + student + school
// 3. Render PDF with @react-pdf/renderer
//    (school logo, student name, amounts,
//     receipt number, date, signature line
// 4. Upload to Supabase Storage:
//    fee-receipts/GNPS-2026-27-0042.pdf
// 5. Return signed URL (1-hour expiry)

// Response — 200
{
  "pdf_url": "https://...supabase.co/
    storage/v1/object/sign/...",
  "receipt_number": "GNPS/2026-27/0042"
}
// Browser opens PDF in new tab → print
```

4 View fee defaulters

Filter the ledger for overdue items. This is also displayed on the principal's dashboard as an alert count.

QUERY OVERDUE ENTRIES

```
const { data } = await supabase
  .from('student_fee_ledger')
```

Path A — Direct SDK

accountant principal

```
  .select(`
    balance, due_date,
    fee_head:fee_heads(name),
    student:students(
      admission_number,
      first_name, last_name
    )
  `)
  .eq('status', 'overdue')
  .gt('balance', 0)
  .order('due_date')
```



Exams, Marks & Report Cards

Teachers enter marks, the system computes grades and ranks, principal generates report cards

1 Subject teacher enters marks

A grid view: students down the left, mark columns (theory, practical, internal) across the top. Subject teachers can ONLY enter marks for their assigned subjects — RLS enforces this.

Path A — Direct SDK

subject_teacher class_teacher

BATCH UPSERT MARKS

```
const { error } = await supabase
  .from('student_marks')
  .upsert(
    students.map(s => ({
      student_id: s.id,
      enrollment_id: s.enrollmentId,
      exam_subject_id: examSubjectId,
      theory_marks: s.theory,
      practical_marks: s.practical,
      internal_marks: s.internal,
      // total_marks = GENERATED column
      // (theory + practical + internal)
      is_absent: s.isAbsent,
      entered_by: userId,
      school_id: schoolId
    })),
    { onConflict: 'student_id,exam_subject_
  )

// RLS checks: is this teacher assigned
// to teach this subject in this class?
```

2 Get computed results

SUPABASE.RPC('GET_EXAM_RESULTS')

Total marks, percentage, PSEB grade (A1/A2/B1...), and class rank are computed by a database function — not in JavaScript. This ensures consistency.

Database Function (RPC)

class_teacher principal

```
const { data } = await supabase
  .rpc('get_exam_results', {
    p_exam_type_id: 'exam_halfyearly',
    p_class_id: 'cls_010',
    p_section_id: 'sec_010a'
  })
// Returns per student:
// {
//   student_name: "Harpreet Kaur",
//   subjects: [
//     { name: "Maths", total: 90,
//       max: 100, grade: "A1" },
//     { name: "Punjabi", total: 65,
//       max: 80, grade: "B1" }
//   ],
//   percentage: 91.0,
//   overall_grade: "A1",
//   rank: 3,
//   attendance_pct: 94.2,
//   result: "pass"
// }
```

3 Generate report cards (bulk)

Principal clicks "Generate All". This queues a background job. A Supabase Edge Function generates one PDF per student, zips them, uploads to Storage, and notifies the browser via WebSocket.

Path B — API Route → Edge Function

principal

POST /API/REPORTS/BULK-REPORT-CARDS

```
// Request
{
  "exam_type_id": "exam_halfyearly",
  "class_id": "cls_010",
  "section_id": "sec_010a"
}
// Response — 202 Accepted (async!)
{ "job_id": "job_001", "status": "queued" }

// What happens in background:
// 1. Row inserted in job_queue table
// 2. DB webhook triggers Edge Function
// 3. Edge Function: for each student →
//    query marks + attendance → gen PDF
// 4. ZIP all PDFs → upload to Storage
// 5. Update job status → 'complete'
// 6. Realtime pushes notification:

supabase.channel('jobs')
  .on('postgres_changes', {
    event: 'UPDATE',
    table: 'bulk_imports',
    filter: `id=eq.${jobId}`
  }, (payload) => {
```

```
    // payload.new.status === 'complete'
    // payload.new.result_url → ZIP downloa
  }).subscribe()
```

🔒 **Marks locking:** After the principal verifies marks, they're locked (`is_locked = true`). Once locked, teachers can't edit them. Only the principal can unlock. This prevents accidental changes after report cards are printed.

Bulk Student Import & Year Rollover

Import hundreds of students from Excel, and promote them to the next class at year end

1 Upload Excel file

Admin selects a .xlsx file. The file is NOT uploaded to any server — it's parsed entirely in the browser using SheetJS. Each row is validated with Zod. Invalid rows show in red with error messages.

Client-side only (no API call yet)

BROWSER-SIDE PARSING

```
import * as XLSX from 'xlsx'
import { studentSchema } from '@/lib/valida

// 1. Read Excel in browser
const workbook = XLSX.read(fileBuffer)
const rows = XLSX.utils.sheet_to_json(
  workbook.Sheets[workbook.SheetNames[0]]
)

// 2. Validate each row with Zod
const results = rows.map((row, i) => {
  const parsed = studentSchema.safeParse(ro
  return {
    row: i + 1,
    data: row,
    valid: parsed.success,
    errors: parsed.error?.flatten()
  }
})

// 3. Show preview table
// ✅ Valid rows in green
// ❌ Invalid rows in red with messages
```

2 Submit validated rows

POST /API/STUDENTS/BULK-IMPORT

After fixing errors, admin clicks Import. Only the valid JSON rows are sent to the API route — not the Excel file itself. Server validates again (defense in depth) and batch inserts.

Path B — API Route

principal clerk

```
// Request — validated JSON, not raw Excel
{
  "class_id": "cls_005",
  "section_id": "sec_005a",
  "academic_year_id": "ay_2026",
  "students": [
    { "admission_number": "GNPS-2026-0201",
      "first_name": "Arjun", ... },
    // ... up to 200 rows
  ]
}
// Server: validates again, checks for
// duplicate admission numbers, then
// batch INSERT with service role key.

// Response — 200
{
  "imported": 147,
  "errors": 3,
  "error_details": [
    { "row": 42,
      "field": "admission_number",
      "message": "Duplicate: already exists
  ]
}
```

3 Year-end promotion

When the academic year ends, the principal promotes students. This creates NEW enrollment rows — old data is never deleted. A student's complete history across years is always preserved.

Path B — API Route

principal

POST /API/STUDENTS/PROMOTE

```
{
  "from_year": "ay_2025",
  "to_year": "ay_2026",
  "promotions": [
    { "student_id": "stu_001",
      "to_class_id": "cls_006",
      "status": "promoted" },
    { "student_id": "stu_002",
      "to_class_id": "cls_005",
      "status": "detained" },
    { "student_id": "stu_003",
      "status": "tc_issued" }
      // no new enrollment
  ]
}
// Creates NEW rows in
// student_class_enrollments.
// Old enrollment rows stay untouched.
```

```
// This is why enrollment is a separate
// table, not a column on students.
```

Principal's Dashboard & Realtime

Live stats that update automatically — attendance, fees, alerts

1 Dashboard loads pre-computed stats

Stats are NOT queried live (too slow for 30 tables). Instead, a background job computes them every 15 minutes and stores them in a cache table. Dashboard reads the cache — instant load.

Path A — Direct SDK

principal

READ DASHBOARD CACHE

```
const { data } = await supabase
  .from('dashboard_cache')
  .select('*')
  .single()

// Returns:
// {
//   total_students: 1247,
//   total_staff: 42,
//   today_attendance_pct: 87.3,
//   fee_collected_this_month: 452000,
//   fee_outstanding: 185000,
//   chronic_absentees: 14,
//   fee_defaulters: 23,
//   attendance_by_class: [...],
//   fee_monthly_trend: [...],
//   updated_at: "2026-09-27T08:45:00Z"
// }
```

2 Live updates via WebSocket

After the dashboard loads, it subscribes to Supabase Realtime. When the cache refreshes (every 15 min), the new data is pushed to the browser — no page reload needed.

Supabase Realtime (WebSocket)

SUBSCRIBE TO LIVE UPDATES

```
const channel = supabase
  .channel('dashboard-live')
  .on('postgres_changes', {
    event: 'UPDATE',
    schema: 'public',
    table: 'dashboard_cache',
    filter: `school_id=eq.${schoolId}`
  }, (payload) => {
    // Auto-updates charts and numbers
    setDashboardData(payload.new)
  })
  .subscribe()
```

```
// Cleanup when leaving dashboard:
return () => supabase.removeChannel(channel
```

3 Background cron jobs

PostgreSQL cron jobs run automatically. No server or external scheduler needed — it's built into Supabase.

PG_CRON SCHEDULED JOBS

```
-- Every 15 min: refresh dashboard stats
SELECT cron.schedule('*/15 * * * *',
  $$SELECT compute_dashboard_stats()$$
);

-- Daily 6 AM: mark overdue fee entries
SELECT cron.schedule('0 6 * * *',
  $$UPDATE student_fee_ledger
    SET status = 'overdue'
    WHERE due_date < CURRENT_DATE
    AND status = 'pending'$$
);

-- Monday 7 AM: flag chronic absentees
SELECT cron.schedule('0 7 * * MON',
  $$SELECT flag_chronic_absentees(75)$$
  -- anyone below 75% attendance
);
```

Timetable, Homework & Notices

Daily operations — all use Path A (direct SDK)

1 View weekly timetable

Teachers see "My Schedule" — all their periods across the week. Admins see the full school timetable by class.

Path A — Direct SDK

all roles

QUERY TIMETABLE SLOTS

```
// Teacher's own schedule
const { data } = await supabase
  .from('timetable_slots')
  .select(`
    day_of_week,
    period:period_definitions(
      number, start_time, end_time
    ),
    subject:subjects(name),
    class:classes(name),
    section:sections(name)
```

```
  `)
  .eq('teacher_id', myUserId)
  .order('day_of_week')
  .order('period(number)')
```

2 Post homework

Teacher creates homework with optional file attachment (uploaded to Supabase Storage first).

Path A — Direct SDK

class_teacher subject_teacher

INSERT HOMEWORK + UPLOAD

```
// Upload attachment first (if any)
const { data: file } = await supabase.stora
  .from('homework-attachments')
  .upload(`${classId}/${fileName}`, blob)

// Then create homework entry
await supabase.from('homework').insert({
  class_id: classId,
  section_id: sectionId,
  subject_id: subjectId,
  title: 'Ch.5 Exercise 5.3',
  description: 'Solve Q1-15',
  due_date: '2026-09-30',
  attachment_url: file?.path,
  assigned_by: userId,
  school_id: schoolId
})
```

3 Post a notice

Principal posts notices targeted to all, specific classes, or staff only. Two tables: the notice and its targets.

Path A — Direct SDK

principal vice_principal

CREATE NOTICE + TARGETS

```
// 1. Create the notice
const { data: notice } = await supabase
  .from('notices')
  .insert({
    title: 'PTM on 25th September',
    body: 'All parents requested...',
    audience: 'specific_classes',
    priority: 'normal',
    created_by: userId,
    school_id: schoolId
  }).select().single()

// 2. Set which classes see it
await supabase.from('notice_targets')
  .insert([
    { notice_id: notice.id, class_id: 'cls_
    { notice_id: notice.id, class_id: 'cls_
  ])
```

Database — 30 tables at a glance

Every table has `school_id` for future multi-school support. The star of the schema is `student_class_enrollments` — it connects students to classes per year, and almost every other module references it.

Core (3 tables)

schools
academic_years
profiles

Structure & People (10)

classes · sections · subjects
class_subjects · parents
students · student_class_enrollments
staff · staff_expertise
class_teacher_assignments
subject_teacher_assignments

Attendance (1 + 1 view)

attendance_records
v_monthly_attendance_summary

Fees (5)

fee_heads · fee_structures
student_fee_ledger

fee_payments · fee_payment_items
fee_concessions

Academics (5)

period_definitions
timetable_slots · substitutions
syllabus_topics · homework
calendar_events

Exams (4)

exam_types · exam_subjects
student_marks
pseb_grading_scheme
report_cards

Communication (3)

notices · notice_targets
circulars

System (3)

dashboard_alerts
bulk_imports
audit_log

Security — 3 layers of defense

No single point of failure. Even if one layer is bypassed, the other two catch it.

Layer 1 — Edge Middleware: Runs before any page loads. Checks if the user is logged in and if their role allows accessing this URL. Blocks unauthorized routes entirely.

Layer 2 — API Route Handlers: For Path B calls, the server-side code explicitly checks the user's role before proceeding. Wrong role = 403 Forbidden.

Layer 3 — Row Level Security (RLS): The database itself filters rows. A class teacher's query physically cannot return data from other classes. This works for BOTH Path A and Path B (except service role).

How errors are handled

Consistent error shape across the whole system. The frontend uses TanStack Query for automatic retry and error display.

COMMON ERRORS AND WHAT THEY MEAN

```
// Path A (Supabase SDK) errors:
"PGRST301" → JWT expired (auto-refreshed, just retry)
"42501"    → RLS violation — you don't have access
"23505"    → Duplicate entry (e.g., same admission number)
"23503"    → Referenced data missing (e.g., invalid class_id)

// Path B (API Route) errors — consistent shape:
{ "error": "VALIDATION_ERROR",  "message": "...", "field": "..." }   // 400
{ "error": "UNAUTHORIZED",      "message": "..." }                    // 401
{ "error": "FORBIDDEN",         "message": "..." }                    // 403
{ "error": "NOT_FOUND",         "message": "..." }                    // 404
{ "error": "DUPLICATE",         "message": "...", "existing_id": ".." } // 409
{ "error": "MARKS_LOCKED",      "message": "..." }                    // 422

// Client-side pattern (TanStack Query):
const { data, error, isLoading } = useQuery({
  queryKey: ['students', classId],
  queryFn: async () => {
    const { data, error } = await supabase.from('students').select('...')
    if (error) throw error   // TanStack catches + shows error UI
    return data
  },
  retry: (count, err) => err.code !== '42501' && count < 3
  // Don't retry permission errors, retry others up to 3 times
})
```

Validation — same rules everywhere

Zod schemas are defined once and used in three places: the form (browser), the API route (server), and the bulk import preview. Zero chance of client and server validating differently.

LIB/VALIDATIONS/STUDENT.TS — USED EVERYWHERE

```
import { z } from 'zod'
```

```
export const studentSchema = z.object({
  admission_number: z.string().regex(/^GNPS-\d{4}-\d{4}$/, 'Format: GNPS-YYYY-NNNN'),
  first_name:       z.string().min(2).max(50),
  last_name:        z.string().min(2).max(50),
  dob:              z.string().date(),
  gender:           z.enum(['male', 'female', 'other']),
  blood_group:      z.enum(['A+','A-','B+','B-','AB+','AB-','O+','O-']).optional(),
  transport_mode:   z.enum(['school_bus','private','walking','bicycle']).optional(),
})

export const parentSchema = z.object({
  father_name: z.string().min(2).max(100),
  mother_name: z.string().min(2).max(100),
  phone:       z.string().regex(/^[6-9]\d{9}$/, 'Invalid mobile number'),
  address:     z.string().min(5).max(500),
})

// USED IN:
// 1. React Hook Form → zodResolver(studentSchema)
// 2. API Route → studentSchema.safeParse(body)
// 3. Bulk import → validate each Excel row
```

File storage — 4 buckets

student-photos/

Profile photos, max 500KB each. Compressed in browser before upload. Signed URLs for access.

fee-receipts/

Generated PDF receipts. Written by server only (service role). Accountant gets a 1-hour signed URL.

report-cards/

PDF report cards + ZIP bundles. Written by Edge Functions. Principal downloads via signed URL.

homework-attachments/

Teacher-uploaded worksheets and PDFs. Max 5MB. All authenticated users in the school can read.